

CSCE 647 IMAGE SYNTHESIS · MODULE 03 · LECTURE NOTES

Rays and Intersection

The primitive operation, and the numerical trouble in it.

MODULE OUTCOMES

- Derive ray-sphere and ray-triangle intersection.
- Explain what watertightness means and why it matters.
- Solve the self-intersection problem without a magic epsilon.
- Explain ray differentials and what they are for.
- Design the intersection interface a renderer actually needs.

1 Rays

$$\mathbf{r}(t) = \mathbf{o} + t \mathbf{d}, \quad t \in [t_{\min}, t_{\max}]$$

The interval is part of the ray, not an afterthought. A shadow ray is a ray whose $t_{\max}$ stops just short of the light; self-intersection avoidance historically used $t_{\min}$; and BVH traversal (Module 04) shrinks $t_{\max}$ as closer hits are found, which is where much of its pruning power comes from. Put the interval in the struct from the beginning.

Whether to normalise $\mathbf{d}$ is a genuine design choice. With a normalised direction, t is a distance, which is convenient. With an unnormalised direction, a ray can be transformed into an object's local space by a matrix multiply without renormalising, and t retains its meaning in that space — which matters for instancing. PBRT leaves directions unnormalised for this reason.

1.1 Ray-sphere, and a lesson in floating point

Substituting the ray into the implicit sphere equation gives a quadratic. The formula everyone learned at school is numerically poor:

$$t = (-b \pm \sqrt{b^2 - 4ac}) / 2a$$

When b^2 is much larger than $4ac$ — which happens for grazing rays and for spheres large relative to their distance — the square root is very close to $|b|$, and one of the two roots is computed as the difference of two nearly equal numbers. This is **catastrophic cancellation**: the leading significant digits cancel and the result is dominated by rounding error.

$$q = -\frac{1}{2}(b + \text{sign}(b) \sqrt{b^2 - 4ac}) \quad t_1 = q / a, \quad t_2 = c / q$$

THE TEXTBOOK FORMULA IS CORRECT AND UNUSABLE

Both forms give the same roots in exact arithmetic. In floating point the second is accurate and the first is not, and the difference shows up precisely in the configurations that look most impressive: a planet-scale sphere, a grazing horizon ray, an atmosphere shell.

The symptom is a band of noise or missing intersections near the silhouette that does not go away with more samples, because it is not noise — it is a deterministic precision failure.

This is the first instance of a theme that runs through the whole course: the mathematically clean formulation and the numerically usable one are frequently different, and the difference matters most in the cases you most want to render.

1.2 Ray-triangle

The Möller–Trumbore algorithm solves for the intersection parameter and the barycentric coordinates simultaneously, by applying Cramer's rule to a 3×3 system. It requires no precomputation and stores no plane equation, which matters when a scene has a hundred million triangles and memory bandwidth is the binding constraint.

```
vec3 e1 = v1 - v0, e2 = v2 - v0;
vec3 pv = cross(r.d, e2);
float det = dot(e1, pv);
if (fabs(det) < 1e-12f) return false;    // parallel
float inv = 1.0f / det;
vec3 tv = r.o - v0;
float u = dot(tv, pv) * inv;  if (u < 0 || u > 1)    return false;
vec3 qv = cross(tv, e1);
float v = dot(r.d, qv) * inv;  if (v < 0 || u+v > 1)  return false;
float t = dot(e2, qv) * inv;  if (t < tmin || t > tmax) return false;
```

The barycentric coordinates come out free, and you need them immediately: interpolating vertex normals, texture coordinates, tangents, and any other per-vertex attribute is a barycentric combination. The ordering of the early-outs is deliberate — testing u before computing qv saves a cross product on the majority of misses.

2 Watertightness

TWO TRIANGLES SHARING AN EDGE CAN BOTH MISS THE SAME RAY

Consider two triangles that share an edge exactly, as every pair of adjacent triangles in a closed mesh does, and a ray that passes precisely through that shared edge.

Each triangle's intersection test is evaluated independently, in floating point, using *different* vertex orderings and therefore different roundings. It is entirely possible for both tests to conclude that the ray passes just outside the triangle.

The ray passes through a closed solid. In a path tracer, the escaped ray usually finds the environment light, so the artefact appears as isolated bright pixels scattered along silhouettes and creases — and they move as the camera moves, so they cannot be mistaken for texture.

They are also nearly impossible to debug from the image, because the triangle that failed is not the one you would inspect.

A **watertight** intersection test guarantees this cannot occur. The approach of Woop, Benthin, and Wald (2013) is:

- 1 Transform the ray into a space where its direction is $(0, 0, 1)$, choosing the permutation of axes by the largest-magnitude component of $\mathbf{d}$ so the transform is well conditioned.
- 2 Project the three vertices into the plane perpendicular to the ray and compute three 2D edge functions.
- 3 **Evaluate each edge function in a canonical order determined by the vertex positions themselves**, not by the triangle's winding. Two triangles sharing an edge then compute bitwise identical values for that edge.
- 4 If an edge function evaluates to exactly zero, recompute it in double precision to break the tie consistently.

DETERMINISM, NOT PRECISION, IS WHAT BUYS WATERTIGHTNESS

Step 3 is the whole idea, and it is worth being clear about why it works. The test does not become more *accurate* — the edge function may still be computed with error.

What changes is that both triangles compute the *same* value with the *same* error. So if one concludes the ray is outside, the other necessarily concludes it is inside. Exactly one of them reports a hit, which is precisely the guarantee required.

This is a useful pattern beyond ray tracing: when two computations must agree, making them bitwise identical is often easier and more reliable than making them both accurate.

Cost is about 10% over Möller–Trumbore, which is a bargain for eliminating an entire class of artefact that is otherwise undiagnosable. PBRT uses it by default.

3 Self-intersection

WHY SHADOW ACNE HAPPENS

A path hits a surface at parameter t and computes the hit point as $p = o + t \cdot d$. That computation rounds, so **p is not exactly on the surface** — it is at the nearest representable float, which may be slightly behind it.

The next ray starts at p . If p is behind the surface, the new ray immediately intersects the surface it just left, at t very near zero. For a shadow ray this reports the point as being in its own shadow; for an indirect ray it produces a spurious short path.

The resulting dark speckle is the classic 'shadow acne'. It is **worse far from the world origin**, because the spacing between representable floats grows with magnitude: at a coordinate of 10,000, consecutive floats are about a millimetre apart.

This is why a scene imported with real-world coordinates often renders with acne that the same scene centred at the origin does not.

The usual response is a constant: reject hits with t below some epsilon, typically 10^{-4} . This does not work, and the reason it does not work is instructive. The required epsilon scales with the magnitude of the coordinates, so **no single value is correct across a scene**. Too small and acne reappears in the distant parts; too large and contact shadows detach from the objects casting them, so a chair appears to float.

OFFSET IN UNITS-IN-THE-LAST-PLACE INSTEAD

The approach of Wächter and Binder (Ray Tracing Gems, chapter 6) moves the ray origin along the **geometric** normal by a fixed number of representable floats — by reinterpreting the float's bits as an integer, adding a small constant, and reinterpreting back.

Because the spacing of floats scales with magnitude automatically, **a fixed number of ULPs is a correctly scaled offset at every distance from the origin**, with no tuning and no scene dependence.

Near the origin, where ULP steps become vanishingly small, it falls back to a small absolute offset. The combination is robust over the whole representable range.

Offset along the geometric normal, never the shading normal (§5). The shading normal can point to the wrong side of the actual triangle near a silhouette, and offsetting along it can push the origin *into* the surface, which makes the problem worse rather than better.

4 Ray differentials

	Rasteriser	Ray tracer
How it filters textures	Fragments are shaded in 2×2 quads, so hardware computes screen-space derivatives of the texture coordinates by differencing within the quad. Mip level selection is automatic.	Each ray is independent. There is no neighbouring sample, so nothing indicates how much of the texture the ray covers.

	Rasteriser	Ray tracer
Consequence	You had correct texture filtering in CSCE 641 without thinking about it.	Without differentials, you must use the finest mip level and accept severe aliasing on distant or grazing surfaces.

The fix is to carry two auxiliary rays alongside the main one, offset by one pixel in x and in y . At an intersection, the difference between where the three rays land gives $\partial p / \partial x$ and $\partial p / \partial y$, which convert through the surface parameterisation into $\partial u / \partial x$ and $\partial u / \partial y$ — exactly the quantities a mip level requires.

DIFFERENTIALS SURVIVE SPECULAR BOUNCES AND DIE AT DIFFUSE ONES

Through a mirror reflection or a refraction, the differentials can be propagated analytically, with correction terms accounting for the surface's curvature. A reflection in a convex mirror widens the footprint; a concave one narrows it. Igehy's 1999 derivation handles both.

At a diffuse bounce the concept breaks down. The outgoing direction is sampled at random, so two adjacent camera rays scatter in completely unrelated directions and the 'footprint' of the bundle has no meaning.

What production renderers actually do: use differentials for camera rays and specular chains, where they are correct and valuable, and after the first diffuse bounce switch to a heuristic — a fixed blur, a footprint that widens with path length, or path-space filtering.

This is acceptable because indirect illumination is low frequency: an over-blurred texture lookup on a third bounce is invisible, while the same error on a camera ray would not be.

5 The intersection interface

Field	Used for	Notes
t	Ordering hits; updating the ray interval.	The only field a shadow ray needs.
p	Origin of the next ray.	Must be offset per §3 before reuse.
ng	Which side of the surface you are on; the offset direction.	From the triangle itself , by cross product of the edges.
ns	Shading.	Barycentric interpolation of vertex normals, possibly perturbed by a normal map.
u, v	Texture lookup; derivative computation.	Barycentrics, free from Möller–Trumbore.
$dpdu, dpdv$	Tangent frame.	Needed for anisotropic BSDFs (Module 07) and normal mapping.
$prim$	Material and light lookup.	An index or pointer. Keep the hit record small — it is copied constantly.

GEOMETRIC AND SHADING NORMALS MUST BOTH BE STORED

ng is the true normal of the geometry you hit. It determines which side of the surface the ray is on, and it is the correct direction to offset along.

ns is interpolated, and possibly perturbed by a normal map. It is what the BSDF is evaluated against, and the disagreement between it and **ng** is exactly what makes a coarse mesh look smooth.

Use ns to offset and rays can be displaced to the wrong side of the actual triangle near silhouettes, producing light leaks and acne that no epsilon fixes.

Use ng to shade and every smooth mesh is faceted — you have thrown away the vertex normals.

Both mistakes are common in first renderers, and both disappear permanently if the hit record simply carries both. There is also a subtler issue: a sampled direction can be above the shading hemisphere and below the geometric one, which must be rejected explicitly or it produces energy from nowhere.

5.1 Shadow rays are a different operation

A shadow ray asks a yes/no question: is anything between these two points? It does not need the nearest hit, only *any* hit, so it can return the instant one is found rather than continuing to search for a closer one.

- Implement a separate occluded() entry point. It is typically **2–3× faster** than a full intersection query, and after Module 08 introduces next-event estimation, shadow rays will be the majority of all rays cast.
- Set $t_{\max}$ just short of the light sample so the light's own geometry does not register as an occluder — a classic cause of 'my area light is black'.
- Offset at *both* ends: the surface and the light. The light end has the same self-intersection problem.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
PBRT 4th ed. — Chapter 6, Shapes	Intersection for every primitive type, and §6.8 on managing rounding error — the most careful free treatment of §1 and §3 anywhere.
Woop, Benthin & Wald — Watertight Ray/Triangle Intersection (free)	The source for §2. Short paper, complete algorithm, and the failure analysis is worth reading on its own.
Ray Tracing Gems — Chapter 6, A Fast and Robust Method for Avoiding Self-Intersection (free)	The ULP-offset method of §3, with the full code. The whole book is free.
Igehy — Tracing Ray Differentials (1999, free)	The original derivation of §4, including propagation through reflection and refraction.

Resource	What it covers for this module
Ray Tracing in One Weekend	Work through this now if you have not. It gets an image on screen in a few hours and makes the rest of the course concrete.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement ray–sphere intersection with the naive quadratic formula and with the stable form. Construct a case where they differ visibly and report the ratio of the errors.
- 2 Implement Möller–Trumbore and validate the barycentrics by interpolating a known linear function across the triangle and comparing against its analytic value.
- 3 Build a closed cube from twelve triangles and fire ten million random rays at it from outside. Count how many escape. With a naive test the count should be small but non-zero; implement the watertight test and confirm it reaches exactly zero.
- 4 Reproduce the self-intersection problem: place a plane at the origin and render it, then translate the whole scene by 10,000 units and render again with the same epsilon. Report the difference.
- 5 Implement the ULP offset and repeat the previous exercise. Confirm the artefact is gone at both scales without changing any constant.
- 6 Implement ray differentials for camera rays and use them to select mip levels. Render a textured ground plane extending to the horizon, with and without, and compare.
- 7 Measure the speedup of a dedicated occluded() path against calling the full intersection routine, on a scene with significant occlusion. Report the ratio.
- 8 Construct a case where the sampled direction is above the shading hemisphere but below the geometric one. Show what happens if it is not rejected.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why should the ray carry its own parameter interval?
- 2 What is catastrophic cancellation, and where does it occur in ray–sphere intersection?
- 3 What does Möller–Trumbore return beyond t , and why do you need it?
- 4 What does watertightness guarantee, and what mechanism provides it?
- 5 Why is a fixed epsilon the wrong solution to self-intersection?
- 6 How does a ULP-based offset adapt to scene scale automatically?

- 7 What are ray differentials for, and at what point in a path do they stop being meaningful?
- 8 Give the two distinct bugs that follow from conflating geometric and shading normals.
- 9 Why is a shadow ray faster than a general intersection query?